CUDA N-Body Simulation

A GPU-accelerated n-body gravity simulation implemented in CUDA and C++. This project demonstrates how to parallelize the gravitational force calculation across many bodies and update their positions and velocities over time.

The current implementation uses a direct $O(N^2)$ force evaluation, where each body interacts with every other body. That makes it a good teaching example for understanding CUDA kernels, memory traffic, and GPU performance tuning.

Project Goals

This project is intended to:

- model gravitational attraction between many bodies
- offload the expensive computation to the GPU
- provide a simple, readable CUDA reference implementation
- serve as a starting point for optimization experiments and larger physics simulations

Simulation Model

Each body stores:

- position
- velocity
- mass

The force on a body is computed using Newtonian gravity with a softening term to avoid singularities when bodies become too close:

$$\mathbf{a}_i = G \sum_{j \neq i} m_j \frac{\mathbf{r}_j - \mathbf{r}_i}{\left(\|\mathbf{r}_j - \mathbf{r}_i\|^2 + \epsilon^2\right)^{3/2}}$$

where:

- $G = 1.0$
- $\epsilon^2 = 10^{-4}$
- m_j is the mass of body j
- $\mathbf{r}_i$ and $\mathbf{r}_j$ are the positions of bodies i and j

The simulation integrates the acceleration with a simple explicit Euler update:

$$\mathbf{v}_{t+1} = \mathbf{v}_t + \mathbf{a}\Delta t$$

$$\mathbf{r}_{t+1} = \mathbf{r}_t + \mathbf{v}_{t+1}\Delta t$$

Repository Structure

- CMakeLists.txt — CUDA project configuration and build settings

- include/nbody.cuh — shared constants and kernel declarations
- src/nbody.cu — core CUDA kernels for acceleration and integration
- src/main.cu — CPU-side setup, memory allocation, kernel launches, and output
- build/ — generated build artifacts from CMake

Requirements

To build and run this project, you need:

- NVIDIA GPU with CUDA support
- CUDA Toolkit installed and available in your environment
- CMake 3.18 or newer
- A compatible C++ compiler
- On Windows, Visual Studio 2022 with CUDA support is a common option

Build Instructions

From the project root, configure and build the project:

```
cmake -S . -B build -G "Visual Studio 17 2022"
cmake --build build --config Release
```

This generates the Visual Studio solution and builds the `nbody` executable in the build tree.

If you already have a generated solution, you can also open the project in build/NBodyCUDA.sln and build it from there.

Running the Simulation

Run the compiled binary from the appropriate output directory:

```
./build/Release/nbody.exe
```

or:

```
./build/Debug/nbody.exe
```

The program prints setup information such as:

- number of bodies
- threads per block
- number of blocks
- number of simulation steps

It then prints the first several final positions after the simulation completes.

Default Parameters

The current defaults are defined in src/main.cu:

- `N = 8192` bodies
- `steps = 1000`
- `dt = 0.001f`
- `threads = 256` per block

These values are easy to modify for experimentation.

Performance Notes

This is a direct-force n-body implementation, so the runtime grows quadratically with particle count:

- time complexity: $O(N^2)$
- memory traffic: high, because each body reads the positions of all other bodies

This makes it ideal for:

- learning GPU parallel patterns
- benchmarking CUDA kernels
- testing force computation and simulation behavior on moderate body counts

For very large systems, a more advanced approximation method such as Barnes-Hut or a tree-based algorithm would be more scalable.

Optimization and Tuning

This project is a useful benchmark for experimenting with CUDA optimization strategies, including:

- varying block size
- reducing global-memory reads
- using shared memory for tiled computations
- reducing synchronization and launch overhead
- improving numerical stability and step size choices

The benchmark table below reflects a sample optimization pass and is useful for comparing different CUDA launch configurations.

Block Size Benchmark

Threads / Block	Blocks	GPU Time
32	256	503.284 ms
64	128	478.162 ms
128	64	479.112 ms
256	32	460.247 ms
521	16	552.347 ms

Shared-Memory Improvement

Kernel Type	Threads / Block	GPU Time
Original global-memory kernel	256	521.351 ms
Shared-memory tiled kernel	256	466.44 ms average
Shared-memory tiled kernel	256	460.247 ms best

The best configuration observed in this setup was 256 threads per block, and the shared-memory optimization reduced runtime by roughly 10.5% relative to the original implementation.

Important Implementation Details

- GPU memory is allocated using `cudaMalloc`
- body data is copied to the GPU with `cudaMemcpy`
- each thread handles one body
- the acceleration kernel computes the net force for that body
- the integration kernel updates velocity and position
- `cudaGetLastError()` and `cudaDeviceSynchronize()` are used to validate launches and execution

Common Troubleshooting

CUDA not found during build

- confirm the NVIDIA CUDA Toolkit is installed
- verify your compiler and Visual Studio installation includes CUDA support
- ensure `nvcc` is available on the system path

Runtime issues

- check that the system has a compatible NVIDIA GPU and driver
- reduce the number of bodies if running on lower-end hardware
- confirm the correct build configuration matches the installed CUDA environment

Stability concerns

The simulation is intentionally simple and may become numerically noisy if the time step is too large or the initial conditions are extreme. Reducing `dt` generally improves stability.

Potential Extensions

This project is a natural base for adding:

- a CPU reference implementation for validation
- a Barnes-Hut approximation for larger systems
- advanced integration schemes
- collision handling
- visualization or output plotting
- benchmarking scripts for multiple GPU configurations

License

This repository does not currently include a formal license file. If you plan to distribute or publish it, add an explicit license before sharing it more broadly.

Summary

This repository is a compact CUDA learning project that demonstrates how to compute gravitational interactions in parallel on the GPU. It is useful as an educational sample, a benchmark for GPU optimization, and a starting point for more sophisticated n-body simulations.